

1. A) Write a C program to print address, values, and size of the different data types.

```
A)#include <stdio.h>
```

```
int main() {
```

```
    int intVar = 10;
```

```
    float floatVar = 20.5;
```

```
    double doubleVar = 30.1234;
```

```
    char charVar = 'A';
```

```
    printf("intVar: Address = %p, Value = %d, Size = %zu bytes\n",  
(void*)&intVar, intVar, sizeof(intVar));
```

```
    printf("floatVar: Address = %p, Value = %f, Size = %zu bytes\n",  
(void*)&floatVar, floatVar, sizeof(floatVar));
```

```
    printf("doubleVar: Address = %p, Value = %lf, Size = %zu bytes\n",  
(void*)&doubleVar, doubleVar, sizeof(doubleVar));
```

```
    printf("charVar: Address = %p, Value = %c, Size = %zu bytes\n",  
(void*)&charVar, charVar, sizeof(charVar));
```

```
    return 0;
```

```
}
```

B) Write a C program to find the given year is a leap year or not.

```
#include <stdio.h>
```

```
int main() {
```

```
    int year;
```

```
    printf("Enter a year: ");
```

```
    scanf("%d", &year);
```

```

if ((year % 4 == 0 && year % 100 != 0) || (year % 400 == 0))

    printf("%d is a leap year.\n", year);

else

    printf("%d is not a leap year.\n", year);

return 0;

}

```

C) Write a C program swap two numbers using functions (call by value).

```

#include <stdio.h>

void swap(int a, int b) {

    int temp = a;

    a = b;

    b = temp;

    printf("Inside swap function: a = %d, b = %d\n", a, b);

}

int main() {

    int x, y;

    printf("Enter two integers: ");

    scanf("%d %d", &x, &y);

    printf("Before swapping: x = %d, y = %d\n", x, y);

    swap(x, y);

    printf("After swapping in main: x = %d, y = %d\n", x, y);

    return 0;

}

```

2. A) Write a C program to print sum and difference of 2 numbers.

```
#include <stdio.h>

int main() {

    int num1, num2;

    int sum, difference;

    printf("Enter two integers: ");

    scanf("%d %d", &num1, &num2);

    sum = num1 + num2;

    difference = num1 - num2;

    printf("Sum: %d\n", sum);

    printf("Difference: %d\n", difference);

    return 0;

}
```

B) Write a C program to find maximum of three numbers using conditional operator.

```
#include <stdio.h>

int main() {

    int num1, num2, num3;

    int max;

    printf("Enter three integers: ");

    scanf("%d %d %d", &num1, &num2, &num3);

    max = (num1 > num2) ?

        ((num1 > num3) ? num1 : num3) :

        ((num2 > num3) ? num2 : num3);

    printf("The maximum of the three numbers is: %d\n", max);

    return 0;

}
```

C) Write a C program multiplication of two matrix using 2-D arrays.

```
#include <stdio.h>
int main() {
    int a[10][10], b[10][10], result[10][10] = {0};
    int row1, col1, row2, col2;
    printf("Enter rows and columns of first matrix: ");
    scanf("%d %d", &row1, &col1);
    printf("Enter elements of first matrix:\n");
    for (int i = 0; i < row1; i++)
        for (int j = 0; j < col1; j++)
            scanf("%d", &a[i][j]);
    printf("Enter rows and columns of second matrix: ");
    scanf("%d %d", &row2, &col2);
    printf("Enter elements of second matrix:\n");
    for (int i = 0; i < row2; i++)
        for (int j = 0; j < col2; j++)
            scanf("%d", &b[i][j]);
    if (col1 != row2) {
        printf("Error! column of first matrix should be equal to row of\nsecond matrix.\n");
        return -1;
    }
    for (int i = 0; i < row1; i++)
        for (int j = 0; j < col2; j++)
            for (int k = 0; k < col1; k++)
                result[i][j] += a[i][k] * b[k][j];
    printf("Resultant matrix:\n");
    for (int i = 0; i < row1; i++) {
        for (int j = 0; j < col2; j++) {
            printf("%d ", result[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

3. A) Write a C program to find area of circle and square.

```
#include <stdio.h>
#define PI 3.14159265358979323846

int main() {
    float radius, side, area_circle, area_square;

    printf("Enter the radius of the circle: ");
    scanf("%f", &radius);

    printf("Enter the side length of the square: ");
    scanf("%f", &side);

    area_circle = PI * radius * radius;

    area_square = side * side;

    printf("Area of the circle: %.2f\n", area_circle);
    printf("Area of the square: %.2f\n", area_square);

    return 0;
}
```

B) Write a C program to generate electricity bill.

```
#include <stdio.h>

int main() {

    int customer_id;

    char customer_name[100];

    float units_consumed, bill_amount;
```

```
printf("Enter Customer ID: ");
scanf("%d", &customer_id);

printf("Enter Customer Name: ");
scanf("%s", customer_name);

printf("Enter Units Consumed: ");
scanf("%f", &units_consumed);

if (units_consumed <= 100) {
    bill_amount = units_consumed * 1.5;
} else if (units_consumed <= 300) {
    bill_amount = 100 * 1.5 + (units_consumed - 100) * 2.0;
} else {
    bill_amount = 100 * 1.5 + 200 * 2.0 + (units_consumed - 300) * 3.0;
}

bill_amount += 50; // Fixed charge

printf("\nElectricity Bill\n");
printf("-----\n");
printf("Customer ID: %d\n", customer_id);
printf("Customer Name: %s\n", customer_name);
printf("Units Consumed: %.2f\n", units_consumed);
printf("Total Bill Amount: Rs. %.2f\n", bill_amount);
```

```
    return 0;
}
```

C) Write a C program to find an element in array (linear search).

```
#include <stdio.h>
```

```
int linearSearch(int arr[], int size, int key) {
    for (int i = 0; i < size; i++) {
        if (arr[i] == key) {
            return i; // Element found, return its index
        }
    }
    return -1; // Element not found, return -1
}
```

```
int main() {
    int n, key, result;

    // Input the number of elements in the array
    printf("Enter the number of elements in the array: ");
    scanf("%d", &n);

    int arr[n];

    // Input the elements of the array
```

```

printf("Enter the elements of the array:\n");

for (int i = 0; i < n; i++) {
    scanf("%d", &arr[i]);
}

// Input the element to search for
printf("Enter the element to search for: ");
scanf("%d", &key);

// Perform linear search
result = linearSearch(arr, n, key);

// Print the result
if (result != -1) {
    printf("Element found at index %d\n", result);
} else {
    printf("Element not found in the array\n");
}

return 0;
}

```

4. A) What is difference between scanf(%s), gets ()?
 Here are the main differences between gets() and scanf():

Security

- `gets()` is insecure and has been deprecated since C11. It does not perform any bounds checking, which can lead to buffer overflow vulnerabilities.
- `scanf()` is generally safer than `gets()`, but it can still be vulnerable to buffer overflow attacks if not used carefully.

Input Handling

- `gets()` reads the entire line of input, including spaces, until it encounters a newline character or the end of the file.
- `scanf()` reads input until it encounters a whitespace character, newline character, or the end of the file.

Format Specifiers

- `gets()` does not support format specifiers.
- `scanf()` supports various format specifiers, such as `%d`, `%f`, `%c`, `%s`, etc.

Return Value

- `gets()` returns a pointer to the input string or `NULL` on error.
- `scanf()` returns the number of successful assignments or `EOF` on error.

Example Usage

Here's an example of using `gets()` and `scanf()`:

```
#include <stdio.h>
```

```
int main() {
```

```
    char str[100];
```

```
    // Using gets()
```

```
    printf("Enter a line of text: ");
```

```
    gets(str);
```

```
    printf("You entered: %s\n", str);
```

```
// Using scanf()

printf("Enter a string: ");

scanf("%s", str);

printf("You entered: %s\n", str);

return 0;}
```

B) Write a C program to check given number is prime or not.

```
#include <stdio.h>
```

```
#include <stdbool.h>
```

```
bool isPrime(int num) {
    if (num <= 1) {
        return false; // 0 and 1 are not prime numbers
    }
    for (int i = 2; i <= num / 2; i++) {
        if (num % i == 0) {
            return false; // If divisible, it is not a prime number
        }
    }
    return true; // If no divisors, it is a prime number
}
```

```
int main() {
    int num;

    printf("Enter a number: ");
```

```
scanf("%d", &num);  
  
if (isPrime(num)) {  
    printf("%d is a prime number.\n", num);  
} else {  
    printf("%d is not a prime number.\n", num);  
}  
  
return 0;  
}
```

C) Write a C program to access array using 1D.

```
#include <stdio.h>  
  
int main() {  
    int n;  
  
    // Input the number of elements in the array  
    printf("Enter the number of elements in the array: ");  
    scanf("%d", &n);  
  
    int arr[n];  
  
    // Input the elements of the array  
    printf("Enter the elements of the array:\n");  
    for (int i = 0; i < n; i++) {
```

```

        scanf("%d", &arr[i]);
    }

    // Print the elements of the array
    printf("The elements of the array are:\n");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    return 0;
}

```

5. A) Write a C program to find area of rectangle and triangle.
#include <stdio.h>

```

int main() {
    float length, width, base, height;
    float area_rectangle, area_triangle;

    // Input dimensions for the rectangle
    printf("Enter the length and width of the rectangle: ");
    scanf("%f %f", &length, &width);

    // Input dimensions for the triangle

```

```
printf("Enter the base and height of the triangle: ");  
  
scanf("%f %f", &base, &height);  
  
// Calculate the area of the rectangle  
area_rectangle = length * width;  
  
// Calculate the area of the triangle  
area_triangle = 0.5 * base * height;  
  
// Print the areas  
printf("Area of the rectangle: %.2f\n", area_rectangle);  
printf("Area of the triangle: %.2f\n", area_triangle);  
  
return 0;  
}
```

B). Write a C program given number is Armstrong or not.
Copy code

```
#include <stdio.h>  
  
#include <math.h>  
  
int main() {  
  
    int num, originalNum, remainder, result = 0, n = 0;  
  
    printf("Enter an integer: ");  
  
    scanf("%d", &num);
```

```

originalNum = num;
while (originalNum != 0) {
    originalNum /= 10;
    n++;
}

originalNum = num;
while (originalNum != 0) {
    remainder = originalNum % 10;
    result += pow(remainder, n);
    originalNum /= 10;
}

if (result == num)
    printf("%d is an Armstrong number.\n", num);
else
    printf("%d is not an Armstrong number.\n", num);

return 0;
}

```

C). Write a C program to display n series of Fibonacci numbers

```
#include <stdio.h>
```

```

void printFibonacciSeries(int n) {
    int a = 0, b = 1, next;

```

```
printf("Fibonacci Series: ");

for (int i = 1; i <= n; ++i) {
    printf("%d ", a);
    next = a + b;
    a = b;
    b = next;
}

printf("\n");
}

int main() {
    int n;

    // Input the number of terms
    printf("Enter the number of terms: ");
    scanf("%d", &n);

    // Print the Fibonacci series
    printFibonacciSeries(n);

    return 0;
}
```

6. A) Write a C program to print conversion of Fahrenheit to Celsius
#include <stdio.h>

```
int main() {  
  
    float fahrenheit, celsius;  
  
    // Input temperature in Fahrenheit  
    printf("Enter temperature in Fahrenheit: ");  
    scanf("%f", &fahrenheit);  
  
    // Convert Fahrenheit to Celsius  
    celsius = (fahrenheit - 32) * 5 / 9;  
  
    // Print the temperature in Celsius  
    printf("Temperature in Celsius: %.2f\n", celsius);  
  
    return 0;  
}
```

B) Write a C program Eliminate duplicate elements from arrays.
#include <stdio.h>

```
void removeDuplicates(int arr[], int *n) {  
  
    int temp[*n];  
  
    int j = 0;
```

```
// Traverse the input array
for (int i = 0; i < *n; i++) {
    int found = 0;

    // Check if the element is already present in the temp array
    for (int k = 0; k < j; k++) {
        if (arr[i] == temp[k]) {
            found = 1;
            break;
        }
    }

    // If the element is not found, add it to the temp array
    if (!found) {
        temp[j++] = arr[i];
    }
}

// Copy the temp array back to the original array
for (int i = 0; i < j; i++) {
    arr[i] = temp[i];
}
```

```
// Update the size of the array

*n = j;
}

int main() {
    int n;

    // Input the number of elements in the array
    printf("Enter the number of elements in the array: ");
    scanf("%d", &n);

    int arr[n];

    // Input the elements of the array
    printf("Enter the elements of the array:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    // Remove duplicates
    removeDuplicates(arr, &n);

    // Print the array after removing duplicates
```

```
printf("Array after removing duplicates:\n");  
  
for (int i = 0; i < n; i++) {  
    printf("%d ", arr[i]);  
}  
  
printf("\n");  
  
return 0;  
}
```

C) Write a C program to transpose of matrix using 2-D arrays.

```
#include <stdio.h>  
  
int main() {  
    int rows, cols;  
  
    // Input the dimensions of the matrix  
    printf("Enter the number of rows and columns of the matrix: ");  
    scanf("%d %d", &rows, &cols);  
  
    int matrix[rows][cols], transpose[cols][rows];  
  
    // Input the elements of the matrix  
    printf("Enter the elements of the matrix:\n");  
    for (int i = 0; i < rows; i++) {
```

```

        for (int j = 0; j < cols; j++) {
            scanf("%d", &matrix[i][j]);
        }
    }

    // Calculate the transpose of the matrix
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            transpose[j][i] = matrix[i][j];
        }
    }

    // Print the transpose of the matrix
    printf("Transpose of the matrix:\n");
    for (int i = 0; i < cols; i++) {
        for (int j = 0; j < rows; j++) {
            printf("%d ", transpose[i][j]);
        }
        printf("\n");
    }

    return 0;
}

```

7. A) Write a C program to print conversion of Fahrenheit to Celsius and vice versa.

```
#include <stdio.h>
```

```
float fahrenheitToCelsius(float fahrenheit) {  
    return (fahrenheit - 32) * 5 / 9;  
}
```

```
float celsiusToFahrenheit(float celsius) {  
    return (celsius * 9 / 5) + 32;  
}
```

```
int main() {  
    float temperature;  
    char choice;  
  
    // Input choice for conversion  
    printf("Enter 'F' to convert Fahrenheit to Celsius or 'C' to convert Celsius to Fahrenheit: ");  
    scanf(" %c", &choice);  
  
    // Perform the conversion based on user's choice  
    if (choice == 'F' || choice == 'f') {  
        printf("Enter temperature in Fahrenheit: ");  
        scanf("%f", &temperature);
```

```

        printf("Temperature in Celsius: %.2f\n",
fahrenheitToCelsius(temperature));

    } else if (choice == 'C' || choice == 'c') {

        printf("Enter temperature in Celsius: ");

        scanf("%f", &temperature);

        printf("Temperature in Fahrenheit: %.2f\n",
celsiusToFahrenheit(temperature));

    } else {

        printf("Invalid choice! Please enter 'F' or 'C'.\n");

    }

return 0;

}

```

B) Write a C program to simulate a calculator using switch case.

```

#include <stdio.h>

int main() {

    char operator;

    double num1, num2;

    double result;

    // Input the operator

    printf("Enter an operator (+, -, *, /): ");

    scanf(" %c", &operator);

```

```
// Input the operands

printf("Enter two operands: ");

scanf("%lf %lf", &num1, &num2);


// Calculate result based on operator
switch (operator) {
    case '+':
        result = num1 + num2;
        break;
    case '-':
        result = num1 - num2;
        break;
    case '*':
        result = num1 * num2;
        break;
    case '/':
        if (num2 != 0) {
            result = num1 / num2;
        } else {
            printf("Error! Division by zero.\n");
            return 1;
        }
}
```

```

        break;

default:

    printf("Error! Invalid operator.\n");

    return 1;

}

// Print the result

printf("Result: %.2f\n", result);

return 0;

}

```

C) Write a C program matrix addition using 2D arrays.
#include <stdio.h>

```

int main() {
    int rows, cols;

    // Input the dimensions of the matrices
    printf("Enter the number of rows and columns of the matrices: ");
    scanf("%d %d", &rows, &cols);

    int matrix1[rows][cols], matrix2[rows][cols], result[rows][cols];

    // Input the elements of the first matrix
    printf("Enter the elements of the first matrix:\n");
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            scanf("%d", &matrix1[i][j]);
        }
    }
}

```

```

// Input the elements of the second matrix
printf("Enter the elements of the second matrix:\n");
for (int i = 0; i < rows; i++) {
    for (int j = 0; j < cols; j++) {
        scanf("%d", &matrix2[i][j]);
    }
}

// Add the two matrices
for (int i = 0; i < rows; i++) {
    for (int j = 0; j < cols; j++) {
        result[i][j] = matrix1[i][j] + matrix2[i][j];
    }
}

// Print the resulting matrix
printf("Resulting matrix after addition:\n");
for (int i = 0; i < rows; i++) {
    for (int j = 0; j < cols; j++) {
        printf("%d ", result[i][j]);
    }
    printf("\n");
}

return 0;
}

```

8.A) Write a C program to find compound interest and simple interest.

```

#include <stdio.h>
#include <math.h>

```

```

int main() {
    float principal, rate, time, simple_interest, compound_interest;

    // Input principal, rate and time
    printf("Enter the principal amount: ");
    scanf("%f", &principal);

```

```

printf("Enter the rate of interest (annual): ");
scanf("%f", &rate);

printf("Enter the time in years: ");
scanf("%f", &time);

// Calculate simple interest
simple_interest = (principal * rate * time) / 100;

// Calculate compound interest
compound_interest = principal * pow((1 + rate / 100), time) -
principal;

// Print the results
printf("Simple Interest: %.2f\n", simple_interest);
printf("Compound Interest: %.2f\n", compound_interest);

return 0;
}

```

B) Write a C program to find smallest number in ARRAY
#include <stdio.h>

```

int main() {
    int n, smallest;

    // Input the number of elements in the array
    printf("Enter the number of elements in the array: ");
    scanf("%d", &n);

    int arr[n];

    // Input the elements of the array
    printf("Enter the elements of the array:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    // Initialize smallest with the first element
    smallest = arr[0];
}

```

```

// Find the smallest element
for (int i = 1; i < n; i++) {
    if (arr[i] < smallest) {
        smallest = arr[i];
    }
}

// Print the smallest element
printf("The smallest element in the array is: %d\n", smallest);

return 0;
}

```

C) Write a C program to simulate a calculator using switch case.

9. A) Write any 4 string functions
 ere are four commonly used string functions in C along with brief explanations and examples:

1. strlen()

- **Purpose:** Returns the length of a string (excluding the null character \0).
- **Prototype:** size_t strlen(const char *str);
- **Example:**

```

c
#include <stdio.h>
#include <string.h>

int main() {
    char str[] = "Hello, World!";
    printf("Length of the string: %zu\n", strlen(str));
    return 0;
}

```

2. strcpy()

- **Purpose:** Copies the content of one string to another.
- **Prototype:** char *strcpy(char *dest, const char *src);
- **Example:**

```

c
#include <stdio.h>
#include <string.h>

```

```
int main() {
    char src[] = "Hello, World!";
    char dest[50];
    strcpy(dest, src);
    printf("Copied string: %s\n", dest);
    return 0;
}
```

3. strcat()

- **Purpose:** Concatenates (appends) one string to the end of another.
- **Prototype:** char *strcat(char *dest, const char *src);
- **Example:**

```
c
#include <stdio.h>
#include <string.h>
```

```
int main() {
    char str1[50] = "Hello, ";
    char str2[] = "World!";
    strcat(str1, str2);
    printf("Concatenated string: %s\n", str1);
    return 0;
}
```

4. strcmp()

- **Purpose:** Compares two strings lexicographically.
- **Prototype:** int strcmp(const char *str1, const char *str2);
- **Example:**

```
c
#include <stdio.h>
#include <string.h>
```

```
int main() {
    char str1[] = "Hello";
    char str2[] = "World";
    if (strcmp(str1, str2) == 0) {
        printf("The strings are equal.\n");
    } else {
        printf("The strings are not equal.\n");
    }
}
```

```
    }  
    return 0;  
}
```

B) Write a C program sum of a 5digit number
#include <stdio.h>

```
int main() {  
    int number, sum = 0, digit;  
  
    // Input a 5-digit number  
    printf("Enter a 5-digit number: ");  
    scanf("%d", &number);  
  
    // Ensure it's a 5-digit number  
    if (number < 10000 || number > 99999) {  
        printf("Error! Please enter a 5-digit number.\n");  
        return 1;  
    }  
  
    // Calculate the sum of the digits  
    while (number != 0) {  
        digit = number % 10; // Get the last digit  
        sum += digit;        // Add it to the sum  
        number /= 10;        // Remove the last digit  
    }  
  
    // Print the result  
    printf("The sum of the digits is: %d\n", sum);  
  
    return 0;  
}
```

C) Write a C program to display given number in words using
switch case.

```
#include <stdio.h>
```

```
int main() {  
    int number;
```

```
// Input the number
printf("Enter a number (0-9): ");
scanf("%d", &number);

// Display the number in words using switch case
switch (number) {
    case 0:
        printf("Zero\n");
        break;
    case 1:
        printf("One\n");
        break;
    case 2:
        printf("Two\n");
        break;
    case 3:
        printf("Three\n");
        break;
    case 4:
        printf("Four\n");
        break;
    case 5:
        printf("Five\n");
        break;
    case 6:
        printf("Six\n");
        break;
    case 7:
        printf("Seven\n");
        break;
    case 8:
        printf("Eight\n");
        break;
    case 9:
        printf("Nine\n");
        break;
    default:
```

```

        printf("Invalid number! Please enter a number between 0 and
9.\n");
        break;
    }

    return 0;
}

```

10. A) What is array. write syntax and declare an array.

An array is a collection of elements of the same type stored at contiguous memory locations. It allows you to store multiple items of the same type using a single variable name. Arrays can be one-dimensional (like a list) or multi-dimensional (like a table).

Syntax to Declare an Array:

```

c
type arrayName[arraySize];

```

- **type:** Data type of the elements (e.g., int, float, char, etc.).
- **arrayName:** Name of the array.
- **arraySize:** Number of elements the array can hold.

Examples of Array Declarations:

1. One-Dimensional Array:

- Declaring an array of integers:

```

c
int numbers[5];

```

- This declares an array named numbers that can hold 5 integers.

2. Initializing an Array:

- Declaring and initializing an array of integers:

```

c
int numbers[5] = {10, 20, 30, 40, 50};

```

- This declares an array named numbers with 5 elements initialized to 10, 20, 30, 40, and 50.

3. Character Array (String):

- Declaring a character array:

```

c
char name[10];

```

- This declares an array named name that can hold 10 characters.

4. Multi-Dimensional Array (2D Array):

- Declaring a two-dimensional array (like a matrix):

C

```
int matrix[3][4];
```

- This declares a 2D array named matrix with 3 rows and 4 columns.

B) C Program to Insert an Element into an Array

C

```
#include <stdio.h>
```

```
int main() {
```

```
    int n, pos, value;
```

```
    // Input the number of elements in the array
```

```
    printf("Enter the number of elements in the array: ");
```

```
    scanf("%d", &n);
```

```
    int arr[n+1];
```

```
    // Input the elements of the array
```

```
    printf("Enter the elements of the array:\n");
```

```
    for (int i = 0; i < n; i++) {
```

```
        scanf("%d", &arr[i]);
```

```
    }
```

```
    // Input the position and value to insert
```

```
    printf("Enter the position to insert the element: ");
```

```
    scanf("%d", &pos);
```

```
    printf("Enter the value to insert: ");
```

```
    scanf("%d", &value);
```

```
    // Shift elements to the right
```

```
    for (int i = n; i >= pos; i--) {
```

```
        arr[i] = arr[i-1];
```

```
    }
```

```
    // Insert the element
```

```
    arr[pos-1] = value;
```

```

// Print the updated array
printf("Array after insertion:\n");
for (int i = 0; i <= n; i++) {
    printf("%d ", arr[i]);
}
printf("\n");

return 0;
}

```

C) C Program to Count Alphabets and Digits, Convert Lowercase to Uppercase

```

c
#include <stdio.h>
#include <ctype.h>

int main() {
    char str[100];
    int i;
    int lowercase = 0, uppercase = 0, consonants = 0, vowels = 0, digits
= 0;

    // Input the string
    printf("Enter a string: ");
    scanf("%s", str);

    // Traverse the string
    for (i = 0; str[i] != '\0'; i++) {
        char ch = str[i];
        if (islower(ch)) {
            lowercase++;
            ch = toupper(ch);
        }
        if (isupper(ch)) {
            uppercase++;
        }
        if (isdigit(ch)) {
            digits++;
        }
    }
}

```

```

    if (ch == 'A' || ch == 'E' || ch == 'I' || ch == 'O' || ch == 'U') {
        vowels++;
    } else if (isalpha(ch)) {
        consonants++;
    }
    str[i] = ch;
}

// Print the results
printf("Lowercase letters: %d\n", lowercase);
printf("Uppercase letters: %d\n", uppercase);
printf("Consonants: %d\n", consonants);
printf("Vowels: %d\n", vowels);
printf("Digits: %d\n", digits);
printf("Converted string: %s\n", str);

return 0;
}

```

11. A) C Program to Display Total Marks and Average Marks

C

```
#include <stdio.h>
```

```

int main() {
    int n, marks, total = 0;
    float average;

    // Input the number of subjects
    printf("Enter the number of subjects: ");
    scanf("%d", &n);

    // Input the marks and calculate total
    printf("Enter the marks for each subject:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &marks);
        total += marks;
    }

    // Calculate average
    average = (float) total / n;
}

```

```

    // Print the results
    printf("Total Marks: %d\n", total);
    printf("Average Marks: %.2f\n", average);

    return 0;
}

```

11. B) C Program for String Length, Concatenation, and Reversal

```

#include <stdio.h>
#include <string.h>

// Function to reverse a string
void reverseString(char str[]) {
    int n = strlen(str);
    for (int i = 0; i < n / 2; i++) {
        char temp = str[i];
        str[i] = str[n - i - 1];
        str[n - i - 1] = temp;
    }
}

int main() {
    char str1[100], str2[100], str3[100];

    // Input the strings
    printf("Enter the first string: ");
    scanf("%s", str1);
    printf("Enter the second string: ");
    scanf("%s", str2);

    // Calculate and print the length of the first string
    printf("Length of the first string: %zu\n", strlen(str1));

    // Concatenate the two strings
    strcpy(str3, str1); // Copy str1 to str3
    strcat(str3, str2); // Concatenate str2 to str3
    printf("Concatenated string: %s\n", str3);
}

```

```

// Reverse the first string and print it
reverseString(str1);
printf("Reversed first string: %s\n", str1);

return 0;
}

```

C) C Program to Find N Factorial Using Recursion

```

#include <stdio.h>

// Recursive function to calculate factorial
long long int factorial(int n) {
    if (n == 0 || n == 1) {
        return 1;
    } else {
        return n * factorial(n - 1);
    }
}

int main() {
    int n;
    printf("Enter a number to find its factorial: ");
    scanf("%d", &n);

    // Calculate and print the factorial
    printf("Factorial of %d is %lld\n", n, factorial(n));

    return 0;
}

```

12. A) What is a Function and a Pointer, Write Its Syntax

Function:

A function is a block of code that performs a specific task. It can take inputs, process them, and return a result.

- **Syntax:**

```

c
return_type function_name(parameter_list) {
    // function body
}

```

- **Example:**

```
c
int add(int a, int b) {
    return a + b;
}
```

Pointer:

A pointer is a variable that stores the memory address of another variable.

- **Syntax:**

```
c
type *pointer_name;
```

- **Example:**

```
c
int *ptr;
```

B) C Program to Find Min and Max Elements in an Array

```
c
#include <stdio.h>
```

```
int main() {
    int n, min, max;
```

```
    // Input the number of elements in the array
    printf("Enter the number of elements in the array: ");
    scanf("%d", &n);
```

```
    int arr[n];
```

```
    // Input the elements of the array
    printf("Enter the elements of the array:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
```

```
    // Initialize min and max
    min = arr[0];
    max = arr[0];
```

```
    // Find the min and max elements
    for (int i = 1; i < n; i++) {
```

```

        if (arr[i] < min) {
            min = arr[i];
        }
        if (arr[i] > max) {
            max = arr[i];
        }
    }

    // Print the min and max elements
    printf("The minimum element in the array is: %d\n", min);
    printf("The maximum element in the array is: %d\n", max);

    return 0;
}

```

C) C Program to Find Sum of Array

```

c
#include <stdio.h>

int main() {
    int n, sum = 0;

    // Input the number of elements in the array
    printf("Enter the number of elements in the array: ");
    scanf("%d", &n);

    int arr[n];

    // Input the elements of the array
    printf("Enter the elements of the array:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    // Calculate the sum of the array elements
    for (int i = 0; i < n; i++) {
        sum += arr[i];
    }

    // Print the sum

```

```
printf("The sum of the elements in the array is: %d\n", sum);
```

```
return 0;
```

```
}
```

13. A) What is a Variable? Explain with an Example Program

A variable is a named storage location in memory that can hold a value. Variables allow you to store, retrieve, and manipulate data in a program.

- **Example:**

C

```
#include <stdio.h>
```

```
int main() {
```

```
    int age = 25; // Variable declaration and initialization
```

```
    printf("Age is: %d\n", age); // Output: Age is: 25
```

```
    return 0;
```

```
}
```

B) C Program to Check if a String is a Palindrome

C

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#include <stdbool.h>
```

```
// Function to check if a string is a palindrome
```

```
bool isPalindrome(char str[]) {
```

```
    int n = strlen(str);
```

```
    for (int i = 0; i < n / 2; i++) {
```

```
        if (str[i] != str[n - i - 1]) {
```

```
            return false;
```

```
        }
```

```
    }
```

```
    return true;
```

```
}
```

```
int main() {
```

```
    char str[100];
```

```
    // Input the string
```

```
    printf("Enter a string: ");
```

```

scanf("%s", str);

// Check if the string is a palindrome
if (isPalindrome(str)) {
    printf("%s is a palindrome.\n", str);
} else {
    printf("%s is not a palindrome.\n", str);
}

return 0;
}

```

C) C Program to Perform Bubble Sort

```

c
#include <stdio.h>

// Function to perform Bubble Sort
void bubbleSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

int main() {
    int n;

    // Input the number of elements in the array
    printf("Enter the number of elements in the array: ");
    scanf("%d", &n);

    int arr[n];

    // Input the elements of the array
    printf("Enter the elements of the array:\n");

```

```

for (int i = 0; i < n; i++) {
    scanf("%d", &arr[i]);
}

// Perform Bubble Sort
bubbleSort(arr, n);

// Print the sorted array
printf("Sorted array:\n");
for (int i = 0; i < n; i++) {
    printf("%d ", arr[i]);
}
printf("\n");

return 0;
}

```

14. A) Difference Between break and continue with Examples

- **break:** Terminates the loop or switch statement and transfers execution to the statement immediately following the loop or switch.

- **Example:**

-

```
#include <stdio.h>
```

```

int main() {
    for (int i = 0; i < 10; i++) {
        if (i == 5) {
            break; // Exit the loop when i is 5
        }
        printf("%d ", i);
    }
    return 0;
}

```

- **continue:** Skips the current iteration of the loop and continues with the next iteration.

- **Example:**

```
#include <stdio.h>
```

```

int main() {
    for (int i = 0; i < 10; i++) {
        if (i == 5) {
            continue; // Skip the iteration when i is 5
        }
        printf("%d ", i);
    }
    return 0;
}

```

14. B) C Program to Find Array Element Using Binary Search

c

```
#include <stdio.h>
```

```

int binarySearch(int arr[], int size, int key) {
    int left = 0, right = size - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (arr[mid] == key)
            return mid; // Element found, return its index
        if (arr[mid] < key)
            left = mid + 1;
        else
            right = mid - 1;
    }
    return -1; // Element not found, return -1
}

```

```

int main() {
    int n, key, result;

    // Input the number of elements in the array
    printf("Enter the number of elements in the array: ");
    scanf("%d", &n);

    int arr[n];
    // Input the elements of the array (sorted order)
    printf("Enter the elements of the array in sorted order:\n");
    for (int i = 0; i < n; i++) {

```

```

        scanf("%d", &arr[i]);
    }

    // Input the element to search for
    printf("Enter the element to search for: ");
    scanf("%d", &key);

    // Perform binary search
    result = binarySearch(arr, n, key);

    // Print the result
    if (result != -1) {
        printf("Element found at index %d\n", result);
    } else {
        printf("Element not found in the array\n");
    }

    return 0;
}

```

C) C Program to Find String Length and String Copy without Using String Handling Functions

```

c
#include <stdio.h>

// Function to find the length of a string
int stringLength(char str[]) {
    int length = 0;
    while (str[length] != '\0') {
        length++;
    }
    return length;
}

// Function to copy one string to another
void stringCopy(char destination[], char source[]) {
    int i = 0;
    while (source[i] != '\0') {
        destination[i] = source[i];
        i++;
    }
}

```

```

    }
    destination[i] = '\0'; // Null-terminate the destination string
}

int main() {
    char str1[100], str2[100];

    // Input the string
    printf("Enter a string: ");
    scanf("%s", str1);

    // Calculate and print the length of the string
    int length = stringLength(str1);
    printf("Length of the string: %d\n", length);

    // Copy the string and print the copied string
    stringCopy(str2, str1);
    printf("Copied string: %s\n", str2);

    return 0;
}

```

15. A) Difference Between while and do-while Loop with Example Program

The primary difference between while and do-while loop is that in while, the condition is checked before the loop body executes, whereas in do-while, the condition is checked after the loop body executes.

Example Program

```

c
#include <stdio.h>

int main() {
    int i = 0;

    // while loop example
    printf("Using while loop:\n");
    while (i < 5) {
        printf("%d ", i);
        i++;
    }
}

```

```
printf("\n");
```

```
// do-while loop example
```

```
printf("Using do-while loop:\n");
```

```
i = 0;
```

```
do {
```

```
    printf("%d ", i);
```

```
    i++;
```

```
} while (i < 5);
```

```
printf("\n");
```

```
return 0;
```

```
}
```

B) Swap Two Numbers Using Functions and Pointers (Call by Value and Reference)

Call by Value

C

```
#include <stdio.h>
```

```
void swap(int a, int b) {
```

```
    int temp = a;
```

```
    a = b;
```

```
    b = temp;
```

```
    printf("After swapping (Call by Value): a = %d, b = %d\n", a, b);
```

```
}
```

```
int main() {
```

```
    int a, b;
```

```
    printf("Enter two numbers: ");
```

```
    scanf("%d %d", &a, &b);
```

```
    printf("Before swapping: a = %d, b = %d\n", a, b);
```

```
    swap(a, b);
```

```
    return 0;
```

```
}
```

Call by Reference

C

```
#include <stdio.h>
```

```
void swap(int *a, int *b) {  
    int temp = *a;  
    *a = *b;  
    *b = temp;  
}
```

```
int main() {  
    int a, b;  
    printf("Enter two numbers: ");  
    scanf("%d %d", &a, &b);  
    printf("Before swapping: a = %d, b = %d\n", a, b);  
    swap(&a, &b);  
    printf("After swapping (Call by Reference): a = %d, b = %d\n", a,  
b);  
  
    return 0;  
}
```

C) C Program to Print 1 to N Prime Numbers

c

```
#include <stdio.h>
```

```
#include <stdbool.h>
```

```
bool isPrime(int num) {  
    if (num <= 1) {  
        return false; // 0 and 1 are not prime numbers  
    }  
    for (int i = 2; i <= num / 2; i++) {  
        if (num % i == 0) {  
            return false; // If divisible, it is not a prime number  
        }  
    }  
    return true; // If no divisors, it is a prime number  
}
```

```
int main() {  
    int n;
```

```
// Input the upper limit
printf("Enter the upper limit: ");
scanf("%d", &n);

// Print prime numbers from 1 to n
printf("Prime numbers from 1 to %d are:\n", n);
for (int i = 1; i <= n; i++) {
    if (isPrime(i)) {
        printf("%d ", i);
    }
}
printf("\n");

return 0;
}
```